

8 Representation Learning

In previous chapters, we have largely focused on supervised learning, using training samples that have both features/inputs and corresponding labels/outputs, to learn models that predict labels for new data. A recurring theme, however, has been the importance of *how data is represented*. In [Chapter 5](#), we saw that choosing the right feature transformation can make the difference between a model that works and one that doesn't. In [Chapter 7](#), we saw that convolutional neural networks learn hierarchical features automatically from data. This chapter steps back to ask a broader question: can we learn good representations of data *directly*, and why would we want to?

8.1 What is a representation?

A *representation* is a mapping from raw data to a more compact, structured encoding that captures the essential information while discarding irrelevant detail. Consider a

This example is drawn from experiments by Bartlett (1932) and

photograph of some trash cans in front of a fence. A camera records millions of pixel values, but if asked to draw the scene from memory, you would produce something much simpler: a sketch with labels like “wooden fence,” “grey trash can,” “metal.” Your brain has compressed the raw sensory input into a compact description that retains the *meaning* of the scene.

More formally, given data $x \in \mathbb{R}^d$, a representation is a function $g : \mathbb{R}^d \rightarrow \mathbb{R}^k$ (typically with $k < d$) that maps the data to a new space where downstream tasks — classification, clustering, retrieval, generation — become easier.

What makes a representation *good*? Not all compressions are equally useful — a representation that discards the wrong information is worse than useless. Bengio et al. (2013) identify several desirable properties:

1. **Compact** (*minimal*): the representation uses few dimensions, discarding noise and redundancy.
2. **Explanatory** (*sufficient*): it retains enough information to reconstruct or reason about the original data.

Intraub & Richardson (1989), who showed that human memory stores scene *gist* rather than pixel-level detail.

While representations are often lower-dimensional ($k < d$), this is not required. In many modern systems (e.g., transformers), representations may actually be higher-dimensional, but are structured in a way that makes relevant information easier to extract.

“Generally speaking, a

3. **Disentangled:** different dimensions capture independent factors of variation (e.g., digit identity vs. stroke thickness).
4. **Interpretable:** the dimensions correspond to meaningful concepts.
5. **Useful for downstream tasks:** a good representation makes subsequent problem solving easier.

good representation is one that makes a subsequent learning task easier.” — *Deep Learning*, Goodfellow et al. 2016

At a high level, a good representation is one that *preserves* the information we care about while *simplifying* the structure of the data.

These properties are sometimes in tension — for instance, maximizing compactness may sacrifice explanatory power (e.g. a one-bit representation is maximally compact but explains almost nothing). However, as we saw with overfitting, maximizing explanatory power for the *current* dataset may actually *lower* it for future datasets. Compact representations help with generalization: just as limiting model complexity prevents overfitting, a bottleneck in the representation forces the model to capture the most broadly useful structure rather than dataset-specific noise.

Much of representation learning is about navigating these trade-offs.

Representations as embeddings. Each layer of a neural network maps its input to a vector of *activations*. We can think of this activation vector as an *embedding* — a point in a high-dimensional space where position encodes meaning. It is important to distinguish between the *weights* of a layer and the *activations* it produces: the weights are the learned parameters (the filters in a CNN, the entries of a weight matrix), while the activations are the outputs produced when those weights are applied to a specific input. The embedding of a data point is its activation vector, not the weights themselves — the weights define the *mapping* between embedding spaces, while the activations are the *representation* of a particular input at a given level of abstraction. Thus, representation learning can be viewed as learning a geometry over the dataset, where distances and directions encode semantic relationships.

For example, given an image x , the activations at some intermediate layer form a vector that represents the image

In a CNN, the learned filters are the “alphabet” or building blocks of the representation. The embedding of a specific image is the vector of filter activations — how strongly each filter responds at each location. Two images of the same object will produce similar activation patterns even if their pixels differ substantially. (For example, think of the weights as a

dictionary — fixed

at a particular level of abstraction. We might call this “im2vec” by analogy with “word2vec” (discussed in [Section 9.2](#) below). If the network has learned a good representation, then semantically similar images will have embeddings that are close together — two photos of golden retrievers will be nearby in this space, even if the actual pixel values are very different.

8.2 Transfer learning and finetuning

We have already seen representation learning at work, even if we didn’t call it that. When a CNN is trained to classify images, its internal layers learn a hierarchy of increasingly abstract features.

Hierarchical feature learning. We can intuitively understand the types of features learned at different stages by visualizing what individual neurons in a trained CNN respond to. Work by Zeiler and Fergus (2014) showed that early layers learn simple edges and colors, middle layers

combine these into textures and object parts, and the deepest layers respond to entire objects and scenes

activation — fixed after training, they define what the network knows how to detect. The activations are the result of looking something up in that dictionary for a specific input. Two images of the same dog produce similar activations (similar lookup results) even if their pixel values differ substantially.)

deepest layers respond to entire objects and scenes — mirroring the classical computer vision pipeline, except that here the features are *learned* end-to-end from data rather than hand-designed.

Note that in a CNN trained end-to-end for classification, backpropagation is jointly learning *both* the feature representations (what the filters detect) *and* the fully connected weights (how those features are combined for the final task). The two are coupled: the representations that emerge are shaped by the prediction objective.

Neuroscience parallels. This hierarchy shows parallels with what neuroscientists observe in the brain’s visual system, which processes information through a series of areas with increasing complexity — from simple edge detectors in early visual cortex to neurons that respond to whole objects in higher areas. When researchers compare how a deep network and the primate brain each organize visual information, the similarity structures turn out to be closely aligned: images that produce similar neural activation patterns in the brain also produce similar

activation patterns in the deep network, and vice versa.

Yamins et al. (PNAS 2014) compared the similarity structure of deep network activations with neural recordings from primate visual cortex and found a strong correspondence. More recently, [Goldstein et](#)

[al. \(Nature](#)

If a deep network's internal representations capture general-purpose features (edges, textures, object parts), then these representations should be useful for tasks *beyond* the one the network was originally trained on. This is the key insight behind *transfer learning*.

The transfer learning paradigm:

1. **Pretrain** a network on a large dataset for task A (e.g., classifying 1000 object categories on ImageNet), obtaining learned parameters W and b .
2. **Transfer** some or all of these parameters to initialize a new network for task B (e.g., recognizing room types from indoor photographs).
3. **Finetune** the new network on task B's (often much smaller) dataset, resulting in updated parameters W' and b' .

This works because early layers learn general features (edges, textures) that are useful across many visual tasks, while later layers become increasingly task-specific.

Finetuning allows the network to adapt its higher-level

[Communications, 2024](#)) showed that brain embeddings and contextual embeddings from language models share common geometric patterns, suggesting that biological and artificial systems converge on similar representational strategies. The “learned representation” being transferred is precisely the weights and biases of the pretrained network — these encode the feature detectors that the network has discovered.

representations to the new task while preserving the useful lower-level features. When you finetune, the early layers have already mostly “learned” low-level structure, so gradient updates mostly affect the later layers, which adapt to the new task’s semantics. Training from scratch, by contrast, must relearn everything — an expensive and data-hungry process.

Transfer learning is now standard practice in deep learning. Rather than training from scratch, practitioners typically start from a pretrained model and finetune for their specific application. This dramatically reduces the amount of labeled data needed and often yields better results than training from scratch, even with substantial datasets.

In practice, how much of the network to update during finetuning is a design choice. One common strategy is to freeze the early layers entirely (set their gradients to zero) and only train the later layers — this is faster and reduces the risk of overwriting useful low-level features. As a rule of thumb: the more similar task B is to task A, the more layers you can safely freeze.

8.3 Learning representations without labels [🔗](#)

Transfer learning reduces the need for labeled data on the target task, but it still relies on supervised pretraining — someone had to label those million-plus ImageNet images in the first place. This raises a natural question: is all that labeling truly necessary, or can we learn good representations without *any* labels?

There is reason to think we can. The key insight of *self-supervised learning* is that the data itself can provide the supervision signal — no human labels required. And beyond self-supervision, *unlabeled* data is simply abundant: the internet contains billions of images, text documents, and audio recordings, none of which come with labels attached. Learning useful representations from this data is the central question of self-supervised and unsupervised representation learning.

8.3.1 Pretext tasks: from supervised to self-supervised

The idea behind *self-supervised learning* is conceptually simple: instead of requiring human-provided labels, we create a supervised learning problem *from the data itself*, using the structure of the data to generate labels automatically.

The key idea is to design a *pretext task* — an auxiliary prediction problem whose labels are free. By training a network to solve the pretext task, useful representations emerge in its internal layers. This is a natural extension of the supervised learning students are already familiar with — the only difference is *where the labels come from*.

Examples of pretext tasks include:

- **Masked prediction:** Hide part of the input and train the network to predict the missing piece. For images, this might mean masking random patches and predicting their pixel values. For text, this means masking words and predicting them from context (as in BERT).

He et al. (2022) showed that masked autoencoders (MAE) — which mask 75% of image patches and train the network to

- **Context prediction:** Given one part of an input, predict what appears nearby. For example, given one patch of an image, predict which of several candidate patches was its neighbor.
- **Colorization:** Given a grayscale image, predict the colors. To succeed, the network must understand object boundaries, materials, and semantics.

reconstruct them — learn representations that rival or exceed those from supervised pretraining.

In each case, the labels come for free from the data itself — no human annotation required. The representations that emerge are useful *because* solving these tasks requires understanding the structure of the data. A network that can fill in missing image patches must have learned about textures, object boundaries, and spatial layout. A network that can predict masked words must have learned something about syntax and meaning.

Consider a concrete text example. Given the sentence:

After the rain, the grass was ____.

A model trained to predict the missing word must learn

that “wet” or “green” are likely completions — which requires understanding something about rain, grass, and their relationship. This is an example of *self-supervision*: the training signal comes from the data itself, not from human labels.

Self-supervised learning has become the dominant paradigm for training large-scale models. Modern language models are trained by predicting the next word in a sequence — a form of self-supervision that, at scale, produces remarkably capable representations of language.

8.3.2 Context-based learning

Another approach to learning representations without labels relies on *co-occurrence*: elements that appear in similar contexts should have similar representations. This is the principle behind *word embeddings* in natural language processing.

Semantically related words such as “dog” and “cat” should have vectors close together, while unrelated words

like “cat” and “table” should be farther apart. Similarity

like cat and table should be farther apart. Similarity between embeddings is frequently measured using the *inner product* (dot product):

$$a^T b = a \cdot b = \sum_j a_j b_j$$

The inner product indicates how aligned two vectors are: highly positive values imply strong similarity, negative values indicate opposition, and values near zero suggest no similarity (up to a scaling factor related to the magnitude).

A groundbreaking embedding method, *word2vec* (Mikolov et al., 2013), produced embeddings where vector arithmetic corresponds to semantic relationships:

$$\text{embedding}_{\text{paris}} - \text{embedding}_{\text{france}} + \text{embedding}_{\text{italy}} \approx \text{embedding}_{\text{rome}}$$

Such embeddings revealed meaningful semantic relationships like analogies across diverse vocabulary (e.g., *uncle - man + woman ≈ aunt*). The training procedure is a form of self-supervised learning: the network learns to

A related measure is *cosine similarity*, $\frac{a^T b}{\|a\| \|b\|}$, which normalizes for vector magnitude and measures only directional alignment.

predict a word from its context (or vice versa), and the word vectors that emerge from this training capture distributional semantics — the principle that “you shall know a word by the company it keeps.”

The same context-based principle applies far beyond text:

- **Node2Vec** (Grover and Leskovec, 2016) learns embeddings for nodes in a graph by treating random walks as “sentences” and applying a word2vec-like objective. Nodes that appear in similar graph neighborhoods get similar embeddings.
- **Medical embeddings:** Representations of electronic health records (EHRs) embed patient histories into vector spaces where patients with similar conditions cluster together, enabling better clinical prediction models.
- **Image embeddings:** As discussed in [Section 8.1](#), intermediate layers of CNNs produce embeddings where visually and semantically similar images are nearby.

8.3.3 Contrastive and multimodal learning

The methods above — self-supervised pretext tasks and context-based learning — learn representations by predicting or co-locating data. *Contrastive learning* takes a different approach: rather than reconstructing or predicting parts of the input, it learns by *comparing* examples, directly shaping the geometry of the representation space.

The core idea is simple:

1. Take a data point and create two augmented versions of it (e.g., two different crops / color shifts of the same image). These form a *positive pair*.
2. Other data points in the batch form *negative pairs*.
3. Train the network to produce representations where positive pairs are close together and negative pairs are far apart.

This encourages the network to learn representations that are *invariant* to the augmentations (cropping, color changes, etc.) while still distinguishing different inputs. Frameworks like SimCLR and MoCo have shown that

He et al. (2020) introduced MoCo (Momentum Contrast), which maintains a

contrastive learning can produce representations competitive with supervised pretraining.

Multimodal learning. The contrastive principle extends naturally to learning shared representations across different data types — for instance, images and their text descriptions, or audio and video. The intuition is that different modalities provide complementary views of the same underlying concept. A photo of a dog and the sentence “a golden retriever playing fetch” both refer to the same scene. By training a model to pull together representations of matching image-text pairs (positives) while pushing apart mismatched pairs (negatives), we obtain a shared embedding space where related concepts from different modalities are close together, regardless of their original form.

This idea underlies models like CLIP, which learns a joint embedding space for images and text. Given an image, CLIP can retrieve relevant text descriptions, and vice versa — all without being explicitly trained on any specific classification task. The shared representation enables flexible “zero-shot” transfer to new tasks simply by

dynamic queue of negative examples, enabling contrastive learning to scale effectively.

LeCun (2022) argues that learning representations that integrate multiple modalities and enable prediction at multiple

describing them in natural language.

8.4 Autoencoders

A special case of self-supervised learning arises when the pretext task is to reconstruct the input x *itself* — not a masked portion, not a neighboring patch, but the entire input. When this reconstruction is forced through a bottleneck, the result is an *autoencoder*, and the latent representation at the bottleneck becomes very explicit.

In a simple, linearly connected network (without residual connections or separate convolution streams), we can think of the activations at an intermediate layer as the latent representation. The autoencoder makes this idea precise: the network is clamped to produce itself, and the bottleneck enforces an *information constraint* that forces a compressed encoding capturing the essential structure of the data.

Assume that we have input data $\mathcal{D} = \{x^{(1)}, \dots, x^{(n)}\}$,

where (i) d We seek to learn an autoencoder that will

time scales is a key step toward more autonomous machine intelligence.

Zero-shot means the model can handle categories it was never explicitly trained on. For example, given an image and the text label “panda,” CLIP can decide whether they match, even if it never saw a labeled panda during training.

where $x^{(i)} \in \mathbb{R}^d$. We seek to learn an autoencoder that will produce a new dataset $\mathcal{D}_{out} = \{a^{(1)}, \dots, a^{(n)}\}$, where $a^{(i)} \in \mathbb{R}^k$ with $k < d$. We can think about $a^{(i)}$ as the new *representation* of data point $x^{(i)}$. For example, in [Figure 8.1](#) we show the learned representations of a dataset of MNIST digits with $k = 2$. We see, after inspecting the individual data points, that unsupervised learning has found a compressed (or *latent*) representation where images of the same digit are close to each other, potentially greatly aiding subsequent clustering or classification tasks.

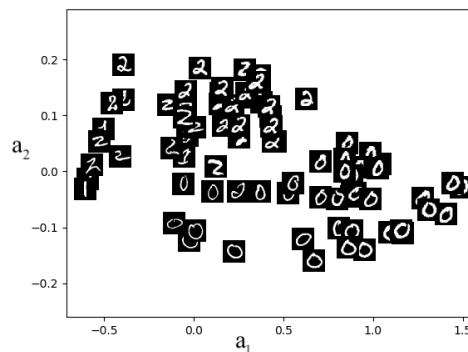


Figure 8.1:
Compression of digits dataset into two dimensions. The input $x^{(i)}$, an image of a handwritten digit, is shown at the new low-dimensional representation (a_1, a_2) .

Structure. An autoencoder consists of two functions: a vector-valued *encoder* $g : \mathbb{R}^d \rightarrow \mathbb{R}^k$ that maps the data to

the representation space $a \in \mathbb{R}^k$, and a *decoder* $h : \mathbb{R}^k \rightarrow \mathbb{R}^d$ that maps the representation space back into the original data space.

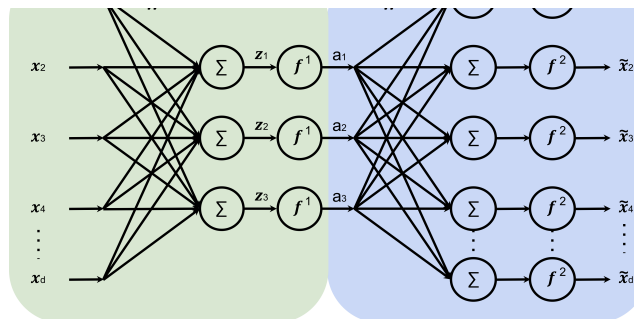
The encoder and decoder are typically neural networks. The basic architecture of a single-layer autoencoder is shown in [Figure 8.2](#); note that bias terms W_0^1 and W_0^2 into the summation nodes exist, but are omitted for clarity in the figure. The original d -dimensional input is compressed into k dimensions via the encoder

$g(x; W^1, W_0^1) = f_1(W^1 T x + W_0^1)$ with $W^1 \in \mathbb{R}^{d \times k}$ and $W_0^1 \in \mathbb{R}^k$, where the non-linearity f_1 is applied element-wise. To recover (an approximation to) the original instance, we then apply the decoder

$h(a; W^2, W_0^2) = f_2(W^2 T a + W_0^2)$, where f_2 denotes a different activation function. In general, both the encoder and decoder could involve multiple layers. Learning seeks parameters W^1, W_0^1 and W^2, W_0^2 such that the reconstructed instances, $h(g(x^{(i)}; W^1, W_0^1); W^2, W_0^2)$, are close to the original input $x^{(i)}$.



Figure 8.2:



Autoencoder structure, showing the encoder (left half, light green), and the decoder (right half, light blue), encoding inputs x to the representation a , and decoding the representation to produce $\tilde{x}$, the reconstruction. In this specific example, the representation (a_1, a_2, a_3) only has three dimensions.

Learning. We learn the weights using the same tools from supervised learning: (stochastic) gradient descent to minimize a loss function. The loss $\mathcal{L}(\tilde{x}, x)$ measures the discrepancy between the reconstruction $\tilde{x} = h(g(x; W^1, W_0^1); W^2, W_0^2)$ and the original input x . For continuous-valued x , a natural choice is squared loss:

Alternatively, you could think of this as *multi-task learning*, where the goal is to predict each dimension of x . One can mix-and-

$$\mathcal{L}_{SE}(\tilde{x}, x) = \sum_{j=1}^d (x_j - \tilde{x}_j)^2.$$

match loss functions as appropriate for each dimension's data type.

Learning then seeks to optimize the parameters of h and g so as to minimize the reconstruction error:

$$\min_{W^1, W_0^1, W^2, W_0^2} \sum_{i=1}^n \mathcal{L}_{SE} \left(h(g(x^{(i)}; W^1, W_0^1); W^2, W_0^2), x^{(i)} \right)$$

The importance of the bottleneck. Without further constraints, it is always possible to perfectly reconstruct the input — for example, by letting $k = d$ and using identity functions. To learn something useful, we must create a *bottleneck* by making k smaller (often much smaller) than d . This forces the learning algorithm to discover transformations that describe the original data using as simple a description as possible. For the digits dataset, a compressed representation might capture the digit label (0–9), rotation, and stroke thickness. After learning, we can inspect the learned representations — for example, by varying one dimension of a and observing how the decoded output $h(a)$ changes — to understand what the autoencoder has learned.

? Study Question

If we set $k = d$ (the bottleneck has the same dimension as the input), what would the autoencoder learn? Why is the bottleneck essential for learning useful representations?

Linear encoders and PCA. Even linear encoders and decoders can be powerful. When both f_1 and f_2 are the identity function, the optimal solution can be obtained in closed form via *principal components analysis* (PCA) using singular value decomposition (SVD). The neural network autoencoders discussed above can be thought of as nonlinear generalizations of PCA, just as a multilayer neural network for regression generalizes linear regression.

From compression to downstream tasks. Once a good encoder has been learned, the decoder can be discarded and the encoder used as a feature extractor. The representations $a = g(x)$ can then be fed into a classifier or regressor trained with supervised learning — potentially using a much smaller labeled dataset. This connects

autoencoders to the semi-supervised learning setting described in [Section 8.3.1](#): we use a large unlabeled dataset to learn the representation and a small labeled dataset for the downstream task.

8.5 Embeddings: a unifying view

The representations learned by all the methods above are, at their core, *embeddings*: mappings from complex, high-dimensional data (images, words, sentences, graphs) into vectors in a lower-dimensional space. The key property of a good embedding is that **geometric relationships in the vector space reflect meaningful relationships in the original data**.

A striking theme across all these methods is that *semantic meaning emerges from structural regularities in the data, not from labels*. An autoencoder trained on digit images, with no knowledge of digit labels, learns a latent space where images of the same digit cluster together. A language model trained to predict the next word, with no explicit

Importantly, embeddings don't need exact coordinates — it's their *relative positioning* within the vector space that matters.

grammar instruction, learns syntactic and semantic structure. A contrastive model trained with only image augmentations learns to distinguish object categories it was never told about. Why? Because *the factors that explain the structure of the data are often the same factors that are semantically meaningful*. The learning algorithms discover these factors not because they are told to, but because doing so is the most efficient way to solve their training objectives. This also helps explain why some pretext tasks work better than others: tasks that require understanding *meaning* (predicting masked words, reconstructing missing image regions) tend to produce better representations than tasks solvable by surface-level shortcuts.

Despite the diversity of methods, they all share a common goal: imposing constraints that reveal latent structure. The differences lie in *what constraint* is used:

Paradigm	Core constraint	Examples
Prediction	Predict missing or future data	Masked language

Table 8.1: Five paradigms for learning representations. Each imposes a different

		models, colorization, next-word prediction (Section 8.3.1)	constraint, but all produce embeddings where geometry encodes meaning.
Context	Co-occurring elements should map nearby	Word2Vec, Node2Vec (Section 8.3.2)	
Contrast	Same input under different views should align	SimCLR, CLIP (Section 8.3.3)	
Compression	Reconstruct through a bottleneck	Autoencoders, PCA (Section 8.4)	
Proximity	Nearby points in data space should stay nearby	t-SNE, UMAP	

These paradigms differ in the strength of their constraints.

Proximity-based methods like t-SNE reveal that structure *exists* by preserving neighborhood relationships, but the resulting embeddings are typically not reusable for new data points. Compression forces the model to retain essential information. Prediction and context require inferring missing semantics. Contrastive learning imposes the strongest constraint, explicitly sculpting the geometry of the embedding space by pulling similar items together and pushing dissimilar items apart.

8.5.1 Ethics of embeddings

Embeddings capture relationships present in their training data — including human biases. Word embeddings trained on internet text have been shown to encode gender stereotypes (e.g., “woman” clusters closer to “nurse” while “man” clusters closer to “doctor”) and other societal biases. Since embeddings form the foundation of many downstream applications, these biases can propagate into real-world decisions.

Detecting and mitigating bias in learned representations is an active area of research, raising important questions

an active area of research, raising important questions

about fairness, accountability, and the social impact of representation learning.

8.5.2 Connection to transformers

The idea of embeddings plays a central role in the transformer architecture discussed in [Chapter 9](#), where word embeddings are dynamically adjusted based on context via the *attention mechanism*. A word like “bank” receives different embeddings depending on whether the surrounding context is about finance or rivers — enabling a more nuanced understanding of language than static embeddings like word2vec can provide.